PROBLEM STATEMENT 01

Build a Coding Agent That Gets the Job Done on the Fewest Tokens

AI coding agents work through long sessions, reading repositories, understanding code, and making dozens of model and tool calls. However, much of this activity is repetitive and expensive. Agents often reread the same files, resend unnecessary context, and use powerful models for tasks that smaller models could handle.

The challenge is to build a coding agent that completes real development tasks correctly while minimizing token consumption, managing costs, and maintaining reliability and security.

GOAL

Build a functional coding agent (CLI, IDE extension, or server) that completes real coding tasks while minimizing token usage without compromising correctness.

MANDATORY REQUIREMENTS

- **Reliability:** Continue working through provider outages or rate limits without losing the session, and inform the user when another model takes over.
- **Code privacy:** Ensure repositories marked confidential are processed only by local models, regardless of instructions contained in prompts.
- **Budget management:** Enforce individual user spending limits before model calls, with clearly defined behavior when a limit is reached.
- **Action approval:** Allow read operations freely, but require explicit approval before writes, shell commands, or Git operations. Prompts must not override these restrictions.

EXPECTED OUTCOME

A working coding agent that demonstrates successful task completion, reduced token consumption, reliable model failover, enforced spending limits, and secure execution of development operations.

OPTIONAL IMPLEMENTATION GUIDANCE

Consider caching repeated requests, routing simpler tasks to smaller models, reducing unnecessary tool descriptions, or remembering previously inspected files. Bifrost can support gateway-level routing, caching, and request management; participants are responsible for implementing the agent's own behavior and security controls.